Modeling State Machines

1 Introduction

In a combinational circuit, outputs depend only on the inputs currently applied; the circuit has no memory of earlier inputs. In a sequential circuit, outputs also depend on the circuit's internal state, which stores information about its history. The state changes over time, usually at a clock edge. Since a practical digital circuit has a finite number of possible states, it can be modeled as a finite-state machine (FSM).

An FSM can be represented by a state table, a state-transition diagram, or an Algorithmic State Machine (ASM) chart. A transition diagram labels each arrow with the input condition and, for a Mealy machine, the output produced on that transition.

1.1 A detector for three or more consecutive 1s

The example below detects three or more consecutive 1s in a serial input stream. States A, B, C, and D represent having seen zero, one, two, or at least three consecutive 1s since the most recent 0. A 0 returns the machine to A; once the machine reaches D, further 1s keep it there. The output becomes 1 on the third consecutive 1 and remains 1 while the run continues.

Reset	Present state	Input	Next state	Output
1	–	–	A	0
0	A	0	A	0
0	A	1	B	0
0	B	0	A	0
0	B	1	C	0
0	C	0	A	0
0	C	1	D	1
0	D	0	A	0
0	D	1	D	1

2 Mealy and Moore Machines

A deterministic FSM can be described by the six-tuple

$$(\Sigma,\ \Gamma,\ S,\ s_0,\ \delta,\ \omega),$$

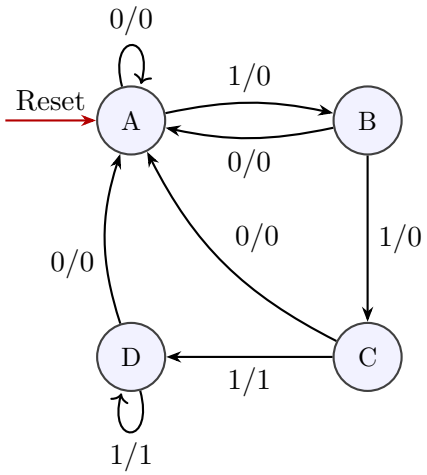


Figure 1: Mealy state diagram for detecting three or more consecutive 1s.

where Σ is the set of input values, Γ is the set of output values, S is the finite set of states, and $s_0 \in S$ is the initial state. The transition function δ selects the next state from the present state and input:

$$\delta : S \times \Sigma \longrightarrow S.$$

The output function distinguishes the two common FSM styles:

- In a **Mealy machine**, $\omega : S \times \Sigma \to \Gamma$; output depends on the present state and current input.
- In a **Moore machine**, $\omega : S \to \Gamma$; output depends only on the present state.

In a typical implementation, next-state logic feeds a bank of flip-flops, whose output is the present state. Mealy output logic sees both the present state and the input, while Moore output logic sees only the present state. This is why a Moore output generally changes only when the registered state changes.

2.1 Cyclic lamp controller

Consider red, green, and yellow lamps that turn on in sequence, advancing once per clock tick. There is no external data input: the clock triggers each transition. This is a Moore machine because the lit lamp is determined by the current state.

The first implementation updates both the state and the lamp outputs in the clocked block. Since the state uses two bits and the three lamp outputs are also registered, synthesis can use five flip-flops. The test bench below toggles the clock and prints the three-bit red-green-yellow output vector.

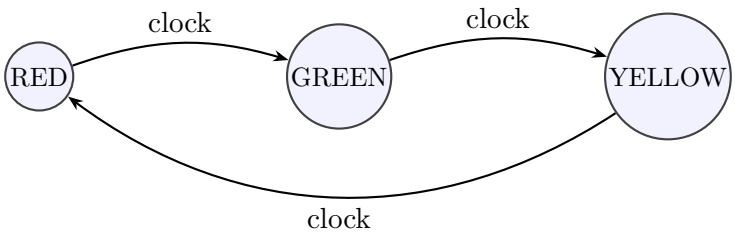


Figure 2: The lamp controller cycles through its three Moore states.

```
module cyclic_lamp (clock, light);
  input clock;
  output reg [0:2] light;
  parameter S0=0, S1=1, S2=2;
  parameter RED=3'b100, GREEN=3'b010, YELLOW=3'b001;
  reg [0:1] state;

  always @(posedge clock)
    case (state)
      S0: begin                  // S0 means RED
        light <= GREEN; state <= S1;
      end
      S1: begin                  // S1 means GREEN
        light <= YELLOW; state <= S2;
      end
      S2: begin                  // S2 means YELLOW
        light <= RED; state <= S0;
      end
      default: begin
        light <= RED;
        state <= S0;
      end
    endcase
endmodule
```

```
module test_cyclic_lamp;
  reg clk;
  wire [0:2] light;

  cyclic_lamp LAMP (clk, light);
  always #5 clk = ~clk;

  initial begin
    clk = 1'b0;
    #100 $finish;
  end

  initial begin
    $dumpfile ("cyclic.vcd");
    $dumpvars (0, test_cyclic_lamp);
    $monitor ($time, " RGY: %b", light);
  end
endmodule
```

To store only the state, update it in one clocked block and decode the lamp outputs in a separate combinational block. This reduces the state storage to two flip-flops; the output logic is combinational. With the encoding in the table, the unused state 11 is treated as a don't-care during minimization.

State (s_1, s_0)	State code	Light (R, G, Y)
S_0	00	100
S_1	01	010
S_2	10	001
Unused	11	xxx

The minimized output equations shown in the source example are

$$R = \overline{s_0}\,\overline{s_1}, \qquad G = s_0, \qquad Y = s_1.$$

```
module cyclic_lamp (clock, light);
  input clock;
  output reg [0:2] light;
  parameter S0=0, S1=1, S2=2;
  parameter RED=3'b100, GREEN=3'b010, YELLOW=3'b001;
  reg [0:1] state;

  always @(posedge clock)
    case (state)
      S0: state <= S1;
      S1: state <= S2;
      S2: state <= S0;
      default: state <= S0;
    endcase

  always @(state)
    case (state)
      S0: light = RED;
      S1: light = GREEN;
      S2: light = YELLOW;
      default: light = RED;
    endcase
endmodule
```

3 Parity Detector

A serial parity detector tracks whether the number of 1s seen so far is even or odd. It starts in EVEN. An input 0 leaves the state unchanged, while an input 1 toggles between EVEN and ODD. The output is 0 in EVEN and 1 in ODD, so it indicates the parity after the most recently sampled input. This is a Moore machine.

The initial clocked version assigns the output in the state cases. The default branch resets the state but does not assign z, which can leave z holding its previous value if that branch is reached. The following version separates the state update from output decoding, with a defined output for every case.

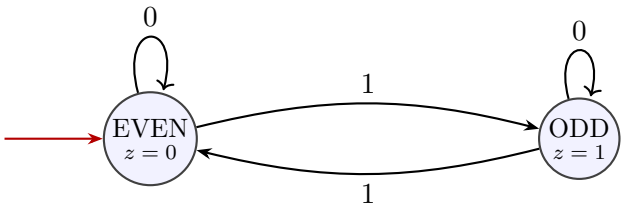


Figure 3: Moore parity detector. Transition labels show the input value.

```
module parity_gen (x, clk, z);
  input x, clk;
  output reg z;
  reg even_odd;                  // The machine state
  parameter EVEN=0, ODD=1;

  always @(posedge clk)
    case (even_odd)
      EVEN: begin
        z <= x ? 1 : 0;
        even_odd <= x ? ODD : EVEN;
      end
      ODD: begin
        z <= x ? 0 : 1;
        even_odd <= x ? EVEN : ODD;
      end
      default: even_odd <= EVEN;
    endcase
endmodule
```

```
module test_parity;
  reg clk, x;
  wire z;

  parity_gen PAR (x, clk, z);
  initial begin
    $dumpfile ("parity.vcd");
    $dumpvars (0, test_parity);
    clk = 1'b0;
  end
  always #5 clk = ~clk;

  initial begin
    #2 x = 0; #10 x = 1; #10 x = 1; #10 x = 1;
    #10 x = 0; #10 x = 1; #10 x = 1; #10 x = 0;
    #10 x = 0; #10 x = 1; #10 x = 1; #10 x = 0;
    #10 $finish;
  end
endmodule
```

```
module parity_gen (x, clk, z);
  input x, clk;
  output reg z;
  reg even_odd;                  // The machine state
  parameter EVEN=0, ODD=1;

  always @(posedge clk)
    case (even_odd)
      EVEN: even_odd <= x ? ODD : EVEN;
      ODD:  even_odd <= x ? EVEN : ODD;
      default: even_odd <= EVEN;
    endcase

  always @(even_odd)
    case (even_odd)
      EVEN: z = 0;
      ODD:  z = 1;
      default: z = 0;
    endcase
endmodule
```

4 Mealy Machine: Sequence Detector

The final example detects the serial pattern 0110. The input x is sampled on clock edges, and output z becomes 1 when the last input bit completes the pattern; otherwise it is 0. Overlapping occurrences are detected by returning to the state that represents the longest useful suffix of the matched pattern. The states record progress through the prefix: S_0 has no partial match, S_1 has matched 0, S_2 has matched 01, and S_3 has matched 011. Since the output is asserted on the transition that completes the pattern, this is a Mealy machine.

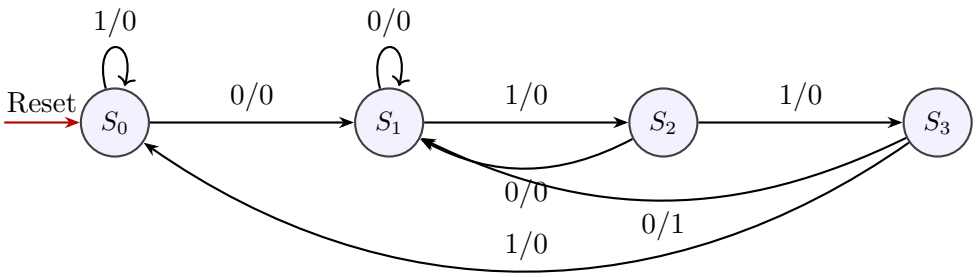


Figure 4: Overlapping 0110 sequence detector. Labels are input/output.

After a match, input 0 leaves a useful suffix of 0, so the machine goes to S_1 rather than resetting completely. This preserves the possibility of detecting an overlapping occurrence.

```
// Sequence detector for pattern "0110"
module seq_detector (x, clk, reset, z);
  input x, clk, reset;
  output reg z;
  parameter S0=0, S1=1, S2=2, S3=3;
  reg [0:1] PS, NS;

  always @(posedge clk or posedge reset)
    if (reset) PS <= S0;
    else       PS <= NS;

  always @(PS, x)
    case (PS)
      S0: begin
        z = x ? 0 : 0;
        NS = x ? S0 : S1;
      end
      S1: begin
        z = x ? 0 : 0;
        NS = x ? S2 : S1;
      end
      S2: begin
        z = x ? 0 : 0;
        NS = x ? S3 : S1;
      end
      S3: begin
        z = x ? 0 : 1;
        NS = x ? S0 : S1;
      end
    endcase
endmodule
```

The test bench asserts reset at the beginning, then applies a sequence of input bits that includes pattern matches and overlapping candidates. The clock period is 10 time units.

```
module test_sequence;
  reg clk, x, reset;
  wire z;

  seq_detector SEQ (x, clk, reset, z);
  initial begin
    $dumpfile ("sequence.vcd");
    $dumpvars (0, test_sequence);
    clk = 1'b0; reset = 1'b1;
    #15 reset = 1'b0;
  end
  always #5 clk = ~clk;

  initial begin
    #12 x = 0; #10 x = 0; #10 x = 1; #10 x = 1;
    #10 x = 0; #10 x = 1; #10 x = 1; #10 x = 0;
    #10 x = 0; #10 x = 1; #10 x = 1; #10 x = 0;
    #10 $finish;
  end
endmodule
```